
RESOURCE ACCESS PROTOCOLS

7.1 INTRODUCTION

A *resource* is any software structure that can be used by a process to advance its execution. Typically, a resource can be a data structure, a set of variables, a main memory area, a file, or a set of registers of a peripheral device. A resource dedicated to a particular process is said to be *private*, whereas a resource that can be used by more tasks is called a *shared resource*. A shared resource protected against concurrent accesses is called an *exclusive resource*.

To ensure consistency of the data structures in exclusive resources, any concurrent operating system should use appropriate resource access protocols to guarantee a mutual exclusion among competing tasks. A piece of code executed under mutual exclusion constraints is called a *critical section*.

Any task that needs to enter a critical section must wait until no other task is holding the resource. A task waiting for an exclusive resource is said to be *blocked* on that resource, otherwise it proceeds by entering the critical section and holds the resource. When a task leaves a critical section, the resource associated with the critical section becomes *free*, and it can be allocated to another waiting task, if any.

Operating systems typically provide a general synchronization tool, called a *semaphore* [Dij68, BH73, PS85], that can be used by tasks to build critical sections. A semaphore is a kernel data structure that, apart from initialization, can be accessed only through two kernel primitives, usually called *wait* and *signal*. When using this tool, each exclusive resource R_k must be protected by a different semaphore S_k and each critical section operating on a resource R_k must begin with a $wait(S_k)$ primitive and end with a $signal(S_k)$ primitive.

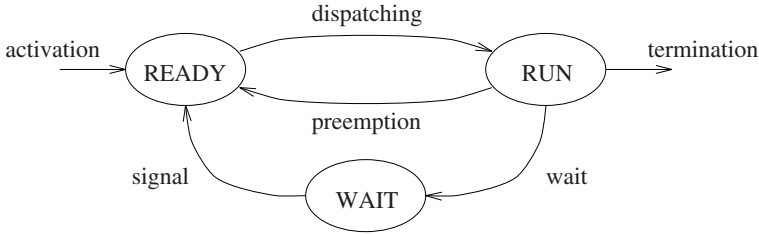


Figure 7.1 Waiting state caused by resource constraints.

All tasks blocked on a resource are kept in a queue associated with the semaphore that protects the resource. When a running task executes a *wait* primitive on a locked semaphore, it enters a *waiting* state, until another task executes a *signal* primitive that unlocks the semaphore. When a task leaves the waiting state, it does not go in the running state, but in the ready state, so that the CPU can be assigned to the highest-priority task by the scheduling algorithm. The state transition diagram relative to the situation described above is shown in Figure 7.1.

In this chapter, we describe the main problems that may arise in a uniprocessor system when concurrent tasks use shared resources in exclusive mode, and we present some resource access protocols designed to avoid such problems and bound the maximum blocking time of each task. We then show how such blocking times can be used in the schedulability analysis to extend the guarantee tests derived for periodic task sets.

7.2 THE PRIORITY INVERSION PHENOMENON

Consider two tasks τ_1 and τ_2 that share an exclusive resource R_k (such as a list) on which two operations (such as *insert* and *remove*) are defined. To guarantee the mutual exclusion, both operations must be defined as critical sections. If a binary semaphore S_k is used for this purpose, then each critical section must begin with a *wait*(S_k) primitive and must end with a *signal*(S_k) primitive (see Figure 7.2).

If preemption is allowed and τ_1 has a higher priority than τ_2 , then τ_1 can be blocked in the situation depicted in Figure 7.3. Here, task τ_2 is activated first, and after a while, it enters the critical section and locks the semaphore. While τ_2 is executing the critical section, task τ_1 arrives, and since it has a higher priority, it preempts τ_2 and starts executing. However, at time t_1 , when attempting to enter its critical section, τ_1 is blocked on the semaphore, so τ_2 resumes. τ_1 has to wait until time t_2 , when τ_2

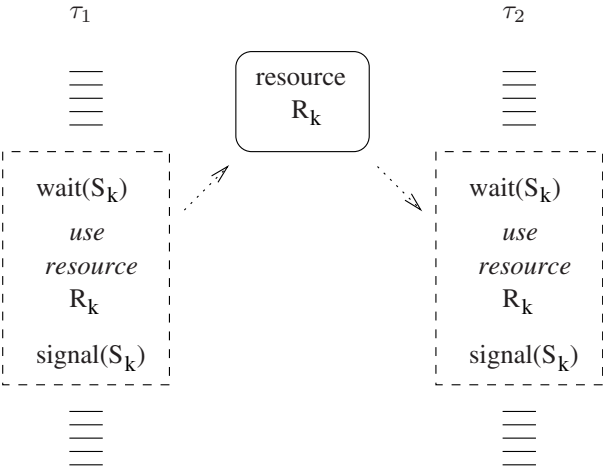


Figure 7.2 Structure of two tasks that share an exclusive resource.

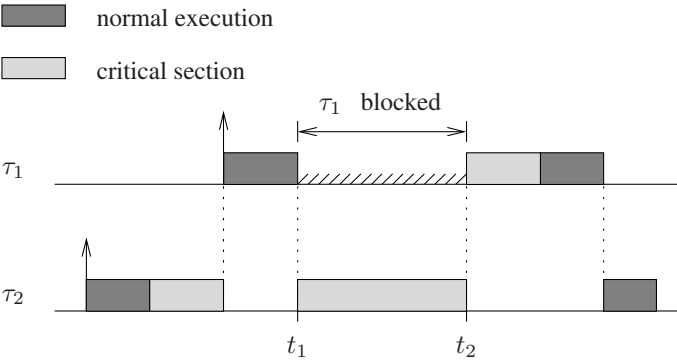


Figure 7.3 Example of blocking on an exclusive resource.

releases the critical section by executing the $\text{signal}(S_k)$ primitive, which unlocks the semaphore.

In this simple example, the maximum blocking time that τ_1 may experience is equal to the time needed by τ_2 to execute its critical section. Such a blocking cannot be avoided because it is a direct consequence of the mutual exclusion necessary to protect the shared resource against concurrent accesses of competing tasks.

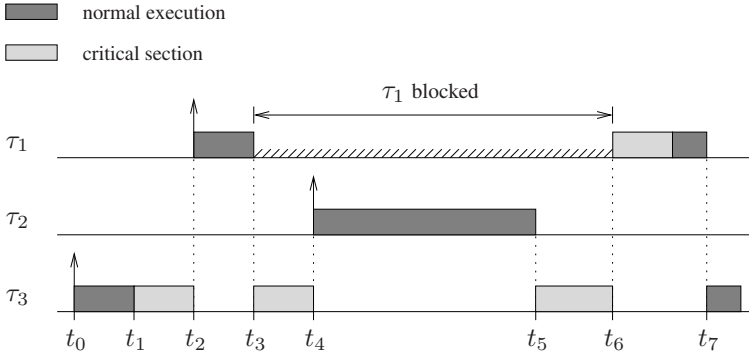


Figure 7.4 An example of priority inversion.

Unfortunately, in the general case, the blocking time of a task on a busy resource cannot be bounded by the duration of the critical section executed by the lower-priority task. In fact, consider the example illustrated in Figure 7.4. Here, three tasks τ_1 , τ_2 , and τ_3 have decreasing priorities, and τ_1 and τ_3 share an exclusive resource protected by a binary semaphore S .

If τ_3 starts at time t_0 , it may happen that τ_1 arrives at time t_2 and preempts τ_3 inside its critical section. At time t_3 , τ_1 attempts to use the resource, but it is blocked on the semaphore S ; thus, τ_3 continues the execution inside its critical section. Now, if τ_2 arrives at time t_4 , it preempts τ_3 (because it has a higher priority) and increases the blocking time of τ_1 by its entire duration. As a consequence, the maximum blocking time that τ_1 may experience does depend not only on the length of the critical section executed by τ_3 but also on the worst-case execution time of τ_2 ! This is a situation that, if it recurs with other medium-priority tasks, can lead to uncontrolled blocking and can cause critical deadlines to be missed. A *priority inversion* is said to occur in the interval $[t_3, t_6]$, since the highest-priority task τ_1 waits for the execution of lower-priority tasks (τ_2 and τ_3). In general, the duration of priority inversion is unbounded, since any intermediate-priority task that can preempt τ_3 will indirectly block τ_1 .

Several approaches have been defined to avoid priority inversion, both under fixed and dynamic priority scheduling.¹ All the methods developed in the context of fixed priority scheduling consist in raising the priority of a task when accessing a shared resource, according to a given protocol for entering and exiting critical sections.

¹Under EDF, such a phenomenon is sometimes referred to as *deadline inversion*.

Similarly, the methods developed under EDF consist in modifying a parameter based on the tasks' relative deadlines.

The rest of this chapter presents the following resource access protocols:

1. Non-Preemptive Protocol (NPP);
2. Highest Locker Priority (HLP), also called Immediate Priority Ceiling (IPC);
3. Priority Inheritance Protocol (PIP);
4. Priority Ceiling Protocol (PCP);
5. Stack Resource Policy (SRP);

Although the first four protocols have been developed under fixed priority assignments, some of them have been extended under EDF.² The Stack Resource Policy, instead, was natively designed to be applicable under both fixed and dynamic priority assignments.

7.3 TERMINOLOGY AND ASSUMPTIONS

Throughout this chapter, we consider a set of n periodic tasks, $\tau_1, \tau_2, \dots, \tau_n$, which cooperate through m shared resources, $R_1, R_2, \dots, R_m$. Each task is characterized by a period T_i and a worst-case computation time C_i . Each resource R_k is guarded by a distinct semaphore S_k . Hence, all critical sections on resource R_k begin with a *wait*(S_k) operation and end with a *signal*(S_k) operation. Since a protocol modifies the task priority, each task is characterized by a fixed *nominal* priority P_i (assigned, for example, by the Rate Monotonic algorithm) and an *active* priority p_i ($p_i \geq P_i$), which is dynamic and initially set to P_i . The following notation is adopted throughout the discussion:

- B_i denotes the maximum blocking time task τ_i can experience.
- $z_{i,k}$ denotes a generic critical section of task τ_i guarded by semaphore S_k .
- $Z_{i,k}$ denotes the longest critical section of task τ_i guarded by semaphore S_k .

²The Priority Inheritance Protocol has been extended for EDF by Spuri [Spu95], and the Priority Ceiling Protocol has been extended for EDF by Chen and Lin [CL90].

- $\delta_{i,k}$ denotes the duration of $Z_{i,k}$.
- $z_{i,h} \subset z_{i,k}$ indicates that $z_{i,h}$ is entirely contained in $z_{i,k}$.
- σ_i denotes the set of semaphores used by τ_i .
- $\sigma_{i,j}$ denotes the set of semaphores that can block τ_i , used by the lower-priority task τ_j .
- $\gamma_{i,j}$ denotes the set of the longest critical sections that can block τ_i , accessed by the lower priority task τ_j . That is,

$$\gamma_{i,j} = \{Z_{j,k} \mid (P_j < P_i) \text{ and } (S_k \in \sigma_{i,j})\} \quad (7.1)$$

- γ_i denotes the set of all the longest critical sections that can block τ_i . That is,

$$\gamma_i = \bigcup_{j: P_j < P_i} \gamma_{i,j} \quad (7.2)$$

Moreover, the properties of the protocols are valid under the following assumptions:

- Tasks $\tau_1, \tau_2, \dots, \tau_n$ are assumed to have different priorities and are listed in descending order of nominal priority, with τ_1 having the highest nominal priority.
- Tasks do not suspend themselves on I/O operations or on explicit synchronization primitives (except on locked semaphores).
- The critical sections used by any task are *properly nested*; that is, given any pair $z_{i,h}$ and $z_{i,k}$, then either $z_{i,h} \subset z_{i,k}$, $z_{i,k} \subset z_{i,h}$, or $z_{i,h} \cap z_{i,k} = \emptyset$.
- Critical sections are guarded by binary semaphores, meaning that only one task at a time can be within a critical section.

7.4 NON-PREEMPTIVE PROTOCOL

A simple solution that avoids the unbounded priority inversion problem is to disallow preemption during the execution of any critical section. This method, also referred to as *Non-Preemptive Protocol* (NPP), can be implemented by raising the priority of a task to the highest priority level whenever it enters a shared resource. In particular, as soon as a task τ_i enters a resource R_k , its dynamic priority is raised to the level:

$$p_i(R_k) = \max_h \{P_h\}. \quad (7.3)$$

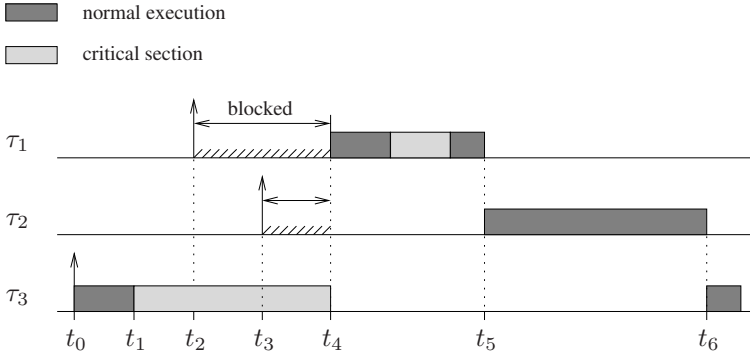


Figure 7.5 Example of NPP preventing priority inversion.

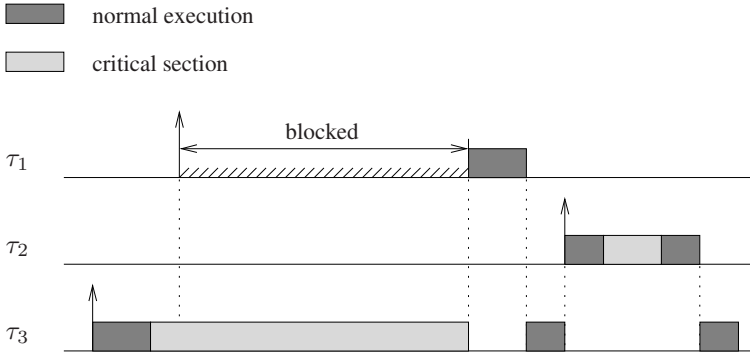


Figure 7.6 Example in which NPP causes unnecessary blocking on τ_1 .

The dynamic priority is then reset to the nominal value P_i when the task exits the critical section. Figure 7.5 shows how NPP solves the priority inversion phenomenon. This method, however, is only appropriate when tasks use short critical sections because it creates unnecessary blocking. Consider, for example, the case depicted in Figure 7.6, where τ_1 is the highest-priority task that does not use any resource, whereas τ_2 and τ_3 are low-priority tasks that share an exclusive resource. If the low-priority task τ_3 enters a long critical section, τ_1 may unnecessarily be blocked for a long period of time.

7.4.1 BLOCKING TIME COMPUTATION

Since a task τ_i cannot preempt a lower priority task τ_j if τ_j is inside a critical section, then τ_i can potentially be blocked by any critical section belonging to a lower priority task. Hence,

$$\gamma_i = \{Z_{j,k} \mid P_j < P_i, k = 1, \dots, m\}$$

Moreover, since a task inside a resource R cannot be preempted, only one resource can be locked at any time t . Hence, a task τ_i can be blocked at most for the length of a single critical section belonging to lower priority tasks. As a consequence, the maximum blocking time τ_i can suffer is given by the duration of the longest critical section among those belonging to lower priority tasks. That is,

$$B_i = \max_{j,k} \{\delta_{j,k} - 1 \mid Z_{j,k} \in \gamma_i\}. \quad (7.4)$$

Note that one unit of time is subtracted from $\delta_{j,k}$ since $Z_{j,k}$ must start before the arrival of τ_i to block it. The unnecessary blocking illustrated in Figure 7.6 can easily be avoided by raising the priority inside a critical section at an opportune level that does not prevent a high priority task to preempt when it is not sharing that resource.

7.5 HIGHEST LOCKER PRIORITY PROTOCOL

The *Highest Locker Priority* (HLP) protocol improves NPP by raising the priority of a task that enters a resource R_k to the highest priority among the tasks sharing that resource. In particular, as soon as a task τ_i enters a resource R_k , its dynamic priority is raised to the level

$$p_i(R_k) = \max_h \{P_h \mid \tau_h \text{ uses } R_k\}. \quad (7.5)$$

The dynamic priority is then reset to the nominal value P_i when the task exits the critical section. The online computation of the priority level in Equation (7.5) can be simplified by assigning each resource R_k a *priority ceiling* $C(R_k)$ (computed off-line) equal to the maximum priority of the tasks sharing R_k ; that is:

$$C(R_k) \stackrel{\text{def}}{=} \max_h \{P_h \mid \tau_h \text{ uses } R_k\}. \quad (7.6)$$

Then, as soon as a task τ_i enters a resource R_k , its dynamic priority is raised to the ceiling of the resource. For this reason, this protocol is also referred to as *Immediate Priority Ceiling*.

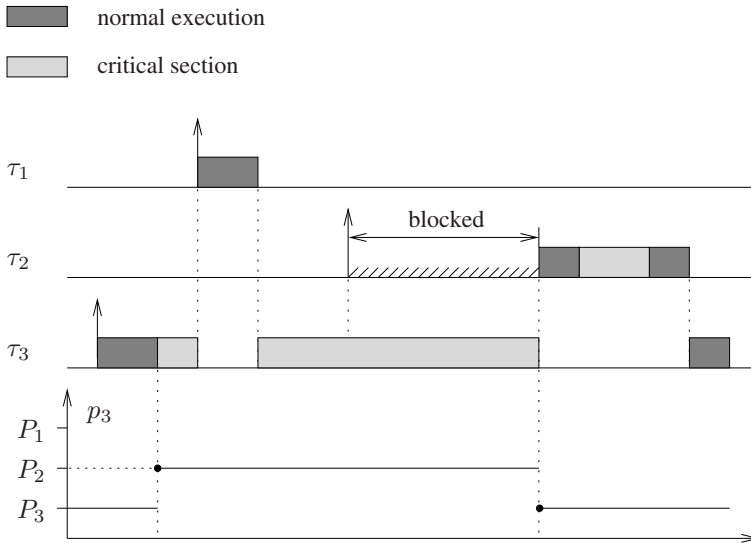


Figure 7.7 Example of schedule under HLP, where p_3 is raised at the level $C(R) = P_2$ as soon as τ_3 starts using resource R .

Note that the schedule produced under HLP for the example generating priority inversion is the same as the one shown in Figure 7.5, since the ceiling of the shared resource is P_1 (the highest priority). Figure 7.7 shows another example in which the ceiling of the shared resource is P_2 ; hence τ_1 can preempt τ_3 inside the critical section, whereas τ_2 is blocked until τ_3 exits its critical section. The figure also shows how the active priority of τ_3 is varied by the protocol during execution.

7.5.1 BLOCKING TIME COMPUTATION

Under HLP, a task τ_i can only be blocked by critical sections belonging to lower priority tasks with a resource ceiling higher than or equal to P_i . Hence,

$$\gamma_i = \{Z_{j,k} \mid (P_j < P_i) \text{ and } C(R_k) \geq P_i\}.$$

Moreover, a task can be blocked at most once, as formally stated in the following theorem.

Theorem 7.1 *Under HLP, a task τ_i can be blocked, at most, for the duration of a single critical section belonging to the set γ_i .*

Proof. The theorem is proved by contradiction, assuming that τ_i is blocked by two critical sections, $z_{1,a}$ and $z_{2,b}$. For this to happen, both critical sections must belong to different tasks (τ_1 and τ_2) with priority lower than P_i , and both must have a ceiling higher than or equal to P_i . That is, by assumption, we must have

$$\begin{aligned} P_1 &< P_i \leq C(R_a); \\ P_2 &< P_i \leq C(R_b). \end{aligned}$$

Now, τ_i can be blocked twice only if τ_1 and τ_2 are both inside the resources when τ_i arrives, and this can occur only if one of them (say τ_1) preempted the other inside the critical section. But, if τ_1 preempted τ_2 inside $z_{2,b}$ it means that $P_2 > C(R_b)$, which is a contradiction. Hence, the theorem follows. $\square$

Since a task τ_i can be blocked at most once, the maximum blocking time τ_i can suffer is given by the duration of the longest critical section among those that can block τ_i . That is,

$$B_i = \max_{j,k} \{ \delta_{j,k} - 1 \mid Z_{j,k} \in \gamma_i \}. \quad (7.7)$$

Note that one unit of time is subtracted from $\delta_{j,k}$ to allow τ_j entering $Z_{j,k}$ before τ_i . This method, although improving NPP, still contains a source of pessimism and could produce some unnecessary blocking. In fact, a task is blocked at the time it attempts to preempt, before it actually requires a resource. If a critical section is contained only in one branch of a conditional statement, then the task could be unnecessarily blocked, since during execution it could take the branch without the resource.

Such a source of pessimism is removed in the Priority Inheritance Protocol by postponing the blocking condition at the entrance of a critical section rather than at the activation time.

7.6 PRIORITY INHERITANCE PROTOCOL

The *Priority Inheritance Protocol* (PIP), proposed by Sha, Rajkumar and Lehoczky [SRL90], avoids unbounded priority inversion by modifying the priority of those tasks that cause blocking. In particular, when a task τ_i blocks one or more higher-priority tasks, it temporarily assumes (*inherits*) the highest priority of the blocked tasks. This prevents medium-priority tasks from preempting τ_i and prolonging the blocking duration experienced by the higher-priority tasks.

7.6.1 PROTOCOL DEFINITION

The Priority Inheritance Protocol can be defined as follows:

- Tasks are scheduled based on their active priorities. Tasks with the same priority are executed in a First Come First Served discipline.
- When task τ_i tries to enter a critical section $z_{i,k}$ and resource R_k is already held by a lower-priority task τ_j , then τ_i is blocked. τ_i is said to be blocked by the task τ_j that holds the resource. Otherwise, τ_i enters the critical section $z_{i,k}$.
- When a task τ_i is blocked on a semaphore, it transmits its active priority to the task, say τ_j , that holds that semaphore. Hence, τ_j resumes and executes the rest of its critical section with a priority $p_j = p_i$. Task τ_j is said to *inherit* the priority of τ_i . In general, a task inherits the highest priority of the tasks it blocks. That is, at every instant,

$$p_j(R_k) = \max\{P_j, \max_h \{P_h | \tau_h \text{ is blocked on } R_k\}\}. \quad (7.8)$$

- When τ_j exits a critical section, it unlocks the semaphore, and the highest-priority task blocked on that semaphore, if any, is awakened. Moreover, the active priority of τ_j is updated as follows: if no other tasks are blocked by τ_j , p_j is set to its nominal priority P_j ; otherwise it is set to the highest priority of the tasks blocked by τ_j , according to Equation (7.8).
- Priority inheritance is transitive; that is, if a task τ_3 blocks a task τ_2 , and τ_2 blocks a task τ_1 , then τ_3 inherits the priority of τ_1 via τ_2 .

EXAMPLES

We first consider the same situation presented in Figure 7.4 and show how the priority inversion phenomenon can be bounded by the Priority Inheritance Protocol. The modified schedule is illustrated in Figure 7.8. Until time t_3 there is no variation in the schedule, since no priority inheritance takes place. At time t_3 , τ_1 is blocked by τ_3 , thus τ_3 inherits the priority of τ_1 and executes the remaining part of its critical section (from t_3 to t_5) at the highest priority. In this condition, at time t_4 , τ_2 cannot preempt τ_3 and cannot create additional interference on τ_1 . As τ_3 exits its critical section, τ_1 is awakened and τ_3 resumes its original priority. At time t_5 , the processor is assigned to τ_1 , which is the highest-priority task ready to execute, and task τ_2 can only start at time t_6 , when τ_1 has completed. The active priority of τ_3 as a function of time is also shown in Figure 7.8 on the lowest timeline.

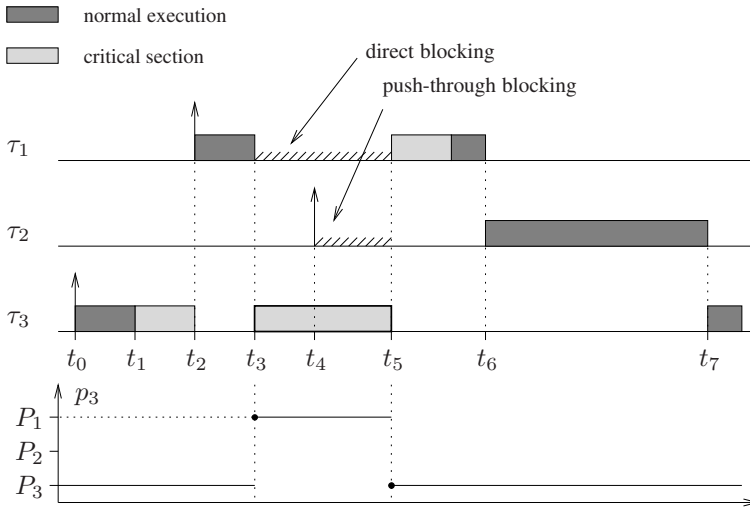


Figure 7.8 Example of Priority Inheritance Protocol.

From this example, we notice that a high-priority task can experience two kinds of blocking:

- **Direct blocking.** It occurs when a higher-priority task tries to acquire a resource already held by a lower-priority task. Direct blocking is necessary to ensure the consistency of the shared resources.
- **Push-through blocking.** It occurs when a medium-priority task is blocked by a low-priority task that has inherited a higher priority from a task it directly blocks. Push-through blocking is necessary to avoid unbounded priority inversion.

Note that in most situations when a task exits a critical section, it resumes the priority it had when it entered. This, however, is not always true. Consider the example illustrated in Figure 7.9. Here, task τ_1 uses a resource R_a guarded by a semaphore S_a , task τ_2 uses a resource R_b guarded by a semaphore S_b , and task τ_3 uses both resources in a nested fashion (S_a is locked first). At time t_1 , τ_2 preempts τ_3 within its nested critical section; hence, at time t_2 , when τ_2 attempts to lock S_b , τ_3 inherits its priority, P_2 . Similarly, at time t_3 , τ_1 preempts τ_3 within the same critical section, and at time t_4 , when τ_1 attempts to lock S_a , τ_3 inherits the priority P_1 . At time t_5 , when τ_3 unlocks semaphore S_b , task τ_2 is awakened but τ_1 is still blocked; hence, τ_3 continues its execution at the priority of τ_1 . At time t_6 , τ_3 unlocks S_a and, since no other tasks are blocked, τ_3 resumes its original priority P_3 .

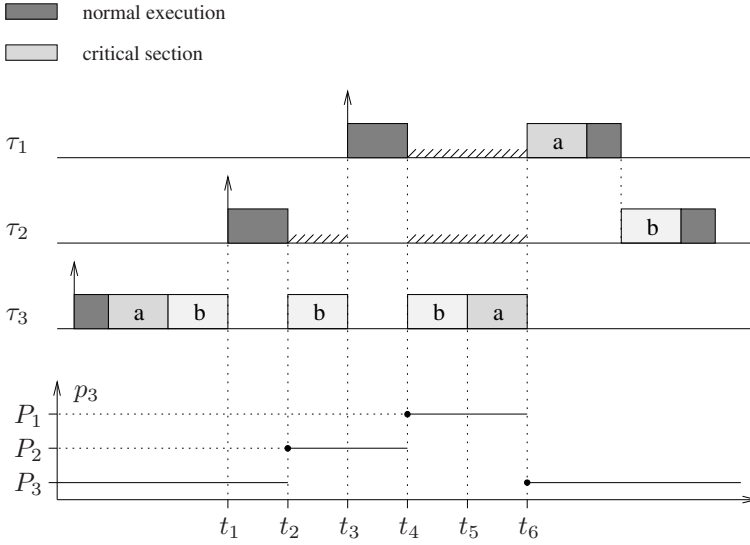


Figure 7.9 Priority inheritance with nested critical sections.

An example of transitive priority inheritance is shown in Figure 7.10. Here, task τ_1 uses a resource R_a guarded by a semaphore S_a , task τ_3 uses a resource R_b guarded by a semaphore S_b , and task τ_2 uses both resources in a nested fashion (S_a protects the external critical section and S_b the internal one). At time t_1 , τ_3 is preempted within its critical section by τ_2 , which in turn enters its first critical section (the one guarded by S_a), and at time t_2 it is blocked on semaphore S_b . As a consequence, τ_3 resumes and inherits the priority P_2 . At time t_3 , τ_3 is preempted by τ_1 , which at time t_4 tries to acquire R_a . Since S_a is locked by τ_2 , τ_2 inherits P_1 . However, τ_2 is blocked by τ_3 ; hence, for transitivity, τ_3 inherits the priority P_1 via τ_2 . When τ_3 exits its critical section, no other tasks are blocked by it; thus it resumes its nominal priority P_3 . Priority P_1 is now inherited by τ_2 , which still blocks τ_1 until time t_6 .

7.6.2 PROPERTIES OF THE PROTOCOL

In this section, the main properties of the Priority Inheritance Protocol are presented. These properties are then used to analyze the schedulability of a periodic task set and compute the maximum blocking time that each task may experience.

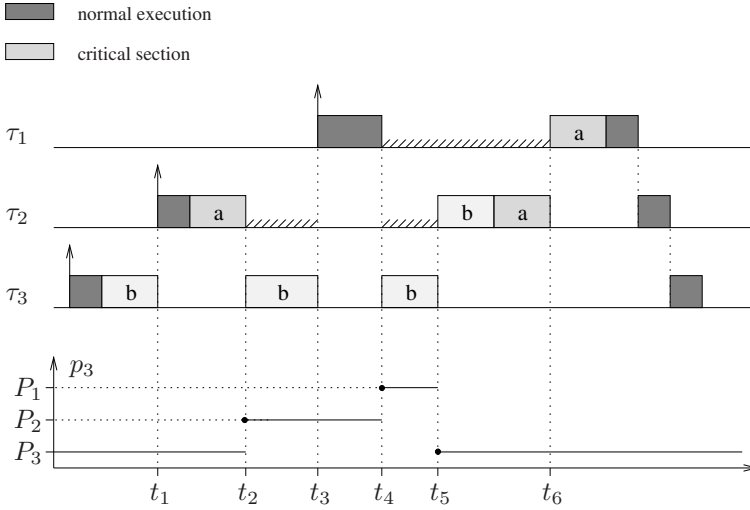


Figure 7.10 Example of transitive priority inheritance.

Lemma 7.1 A semaphore S_k can cause push-through blocking to task τ_i , only if S_k is accessed both by a task with priority lower than P_i and by a task with priority higher than P_i .

Proof. Suppose that semaphore S_k is accessed by a task τ_l with priority lower than P_i , but not by a task with priority higher than P_i . Then, τ_l cannot inherit a priority higher than P_i . Hence, τ_l will be preempted by τ_i and the lemma follows. $\square$

Lemma 7.2 Transitive priority inheritance can occur only in the presence of nested critical sections.

Proof. A transitive inheritance occurs when a high-priority task τ_H is blocked by a medium-priority task τ_M , which in turn is blocked by a low-priority task τ_L (see Figure 7.10). Since τ_H is blocked by τ_M , τ_M must hold a semaphore, say S_a . But τ_M is also blocked by τ_L on a different semaphore, say S_b . This means that τ_M attempted to lock S_b inside the critical section guarded by S_a . The lemma follows. $\square$

Lemma 7.3 *If there are l_i lower-priority tasks that can block a task τ_i , then τ_i can be blocked for at most the duration of l_i critical sections (one for each of the l_i lower-priority tasks), regardless of the number of semaphores used by τ_i .*

Proof. A task τ_i can be blocked by a lower-priority task τ_j only if τ_j has been preempted within a critical section, say $z_{j,k}$, that can block τ_i . Once τ_j exits $z_{j,k}$, it can be preempted by τ_i ; thus, τ_i cannot be blocked by τ_j again. The same situation may happen for each of the l_i lower-priority tasks; therefore, τ_i can be blocked at most l_i times. $\square$

Lemma 7.4 *If there are s_i distinct semaphores that can block a task τ_i , then τ_i can be blocked for at most the duration of s_i critical sections, one for each of the s_i semaphores, regardless of the number of critical sections used by τ_i .*

Proof. Since semaphores are binary, only one of the lower-priority tasks, say τ_j , can be within a blocking critical section corresponding to a particular semaphore S_k . Once S_k is unlocked, τ_j can be preempted and no longer block τ_i . If all s_i semaphores that can block τ_i are locked by the s_i lower-priority tasks, then τ_i can be blocked at most s_i times. $\square$

Theorem 7.2 (Sha, Rajkumar, Lehoczky) *Under the Priority Inheritance Protocol, a task τ_i can be blocked for at most the duration of $\alpha_i = \min(l_i, s_i)$ critical sections, where l_i is the number of lower-priority tasks that can block τ_i and s_i is the number of distinct semaphores that can block τ_i .*

Proof. It immediately follows from Lemma 7.3 and Lemma 7.4. $\square$

7.6.3 BLOCKING TIME COMPUTATION

The evaluation of the maximum blocking time for each task can be computed based on the result of Theorem 7.2. However, a precise evaluation of the blocking factor B_i is quite complex because each critical section of the lower-priority tasks may interfere with τ_i via direct blocking, push-through blocking, or transitive inheritance. In this section, we present a simplified algorithm that can be used to compute the blocking

factors of tasks that do not use nested critical sections. In this case, in fact, Lemma 7.2 guarantees that no transitive inheritance can occur; thus, the analysis of all possible blocking conditions is simplified.

Using the terminology introduced in Section 7.3, in the absence of transitive blocking, the set γ_i of critical sections that can block a task τ_i can be computed as follows:

1. The set of semaphores that can cause direct blocking to τ_i , shared by the lower-priority task τ_j , is

$$\sigma_{i,j}^{\text{dir}} = \sigma_i \cap \sigma_j.$$

2. The set of semaphores that can cause push-through blocking to τ_i , shared by the lower-priority task τ_j , is

$$\sigma_{i,j}^{\text{pt}} = \bigcup_{h: P_h > P_i} \sigma_h \cap \sigma_j.$$

3. The set of semaphores that can block τ_i (either directly or by push-through), shared by the lower-priority task τ_j , is

$$\sigma_{i,j} = \sigma_{i,j}^{\text{dir}} \cup \sigma_{i,j}^{\text{pt}} = \bigcup_{h: P_h \geq P_i} \sigma_h \cap \sigma_j.$$

4. The set of the longest critical sections used by τ_j that can block τ_i (either directly or by push-through) is then

$$\gamma_{i,j} = \{Z_{j,k} \mid (P_j < P_i) \text{ and } (R_k \in \sigma_{i,j})\}.$$

5. The set of all critical sections that can block τ_i (either directly or by push-through) is

$$\gamma_i = \bigcup_{j: P_j < P_i} \gamma_{i,j}.$$

6. B_i is then given by the largest sum of the lengths of the α_i critical sections in γ_i . Note that since τ_i cannot be blocked twice by the same task or by the same semaphore, the sum should contain only terms $\delta_{j,k}$ referring to different tasks and different semaphores.

Unfortunately, even without considering nested resources, the exact computation of each blocking factor requires a combinatorial search for finding the largest sum among all possible α_i durations. A simpler upper bound, however, can be computed according to the following algorithm:

1. According to Lemma 7.3, a task τ_i can be blocked at most once for each of the l_i lower priority tasks. Hence, for each lower priority task τ_j that can block τ_i , sum the duration of the longest critical section among those that can block τ_i ; let B_i^l be this sum. That is,

$$B_i^l = \sum_{j: P_j < P_i} \max_k \{ \delta_{j,k} - 1 \mid Z_{j,k} \in \gamma_i \} \quad (7.9)$$

2. According to Lemma 7.4, a task τ_i can be blocked at most once for each of the s_i semaphores that can block τ_i . Hence, for each semaphore S_k that can block τ_i , sum the duration of the longest critical section among those that can block τ_i ; let B_i^s be this sum. That is,

$$B_i^s = \sum_{k=1}^m \max_j \{ \delta_{j,k} - 1 \mid Z_{j,k} \in \gamma_i \} \quad (7.10)$$

3. According to Theorem 7.2, τ_i can be blocked at most for $\alpha_i = \min(l_i, s_i)$ critical sections. Hence, compute B_i as the minimum between B_i^l and B_i^s . That is,

$$B_i = \min(B_i^l, B_i^s) \quad (7.11)$$

The identification of the critical sections that can block a task can be greatly simplified if for each semaphore S_k we define a *ceiling* $C(S_k)$ equal to the highest-priority of the tasks that use S_k :

$$C(S_k) \stackrel{\text{def}}{=} \max_i \{ P_i \mid S_k \in \sigma_i \}. \quad (7.12)$$

Then, the following lemma holds.

Lemma 7.5 *In the absence of nested critical sections, a critical section $z_{j,k}$ of τ_j guarded by S_k can block τ_i only if $P_j < P_i \leq C(S_k)$.*

Proof. If $P_i \leq P_j$, then task τ_i cannot preempt τ_j ; hence, it cannot be blocked by τ_j directly. On the other hand, if $C(S_k) < P_i$, by definition of $C(S_k)$, any task that uses S_k cannot have a priority higher than P_i . Hence, from Lemma 7.1, $z_{j,k}$ cannot cause push-through blocking on τ_i . Finally, since there are no nested critical sections, Lemma 7.2 guarantees that $z_{j,k}$ cannot cause transitive blocking. The lemma follows.

□

Using the result of Lemma 7.5, the blocking terms B_i^l and B_i^s can be determined as follows:

$$B_i^l = \sum_{j=i+1}^n \max_k \{\delta_{j,k} - 1 \mid C(S_k) \geq P_i\}$$

$$B_i^s = \sum_{k=1}^m \max_{j>i} \{\delta_{j,k} - 1 \mid C(S_k) \geq P_i\}.$$

This computation is performed by the algorithm shown in Figure 7.11. The algorithm assumes that the task set consists of n periodic tasks that use m distinct binary semaphores. Tasks are ordered by decreasing priority, such that $P_i > P_j$ for all $i < j$. Critical sections are not nested. Note that the blocking factor B_n is always zero, since there are no tasks with priority lower than P_n that can block τ_n . The complexity of the algorithm is $O(mn^2)$.

Note that the blocking factors derived by this algorithm are not tight. In fact, B_i^l may be computed by considering two or more critical sections guarded by the same semaphore, which for Lemma 7.4 cannot both block τ_i . Similarly, B_i^s may be computed by considering two or more critical sections belonging to the same task, which for Lemma 7.3 cannot both block τ_i . To exclude these cases, however, the complexity grows exponentially because the maximum blocking time has to be computed among all possible combinations of critical sections. An algorithm based on exhaustive search is presented by Rajkumar [Raj91]. It can find better bounds than those found by the algorithm presented in this section, but it has an exponential complexity.

EXAMPLE

To illustrate the algorithm presented above, consider the example shown in Table 7.1, where four tasks share three semaphores. For each task τ_i , the duration of the longest critical section among those that use the same semaphore S_k is denoted by $\delta_{i,k}$ and reported in the table. $\delta_{i,k} = 0$ means that task τ_i does not use semaphore S_k . Semaphore ceilings are indicated in parentheses.

	$S_a(P_1)$	$S_b(P_1)$	$S_c(P_2)$
τ_1	1	2	0
τ_2	0	9	3
τ_3	8	7	0
τ_4	6	5	4

Table 7.1 Three semaphores shared by four tasks.

Algorithm: Blocking Time Computation**Input:** durations $\delta_{i,k}$ for each task τ_i and each semaphore S_k **Output:** B_i for each task τ_i

// Assumes tasks are ordered by decreasing priorities

```

(1)  begin
(2)    for  $i := 1$  to  $n - 1$  do           // for each task
(3)       $B_i^l := 0$ ;
(4)      for  $j := i + 1$  to  $n$  do           // for each lower priority task
(5)         $D\_max := 0$ ;
(6)        for  $k := 1$  to  $m$  do           // for each semaphore
(7)          if  $(C(S_k) \geq P_i)$  and  $(\delta_{j,k} > D\_max)$  do
(8)             $D\_max := \delta_{j,k}$ ;
(9)          end
(10)        end
(11)         $B_i^l := B_i^l + D\_max - 1$ ;
(12)      end
(13)       $B_i^s := 0$ ;
(14)      for  $k := 1$  to  $m$  do           // for each semaphore
(15)         $D\_max := 0$ ;
(16)        for  $j := i + 1$  to  $n$  do // for each lower priority task
(17)          if  $(C(S_k) \geq P_i)$  and  $(\delta_{j,k} > D\_max)$  do
(18)             $D\_max := \delta_{j,k}$ ;
(19)          end
(20)        end
(21)         $B_i^s := B_i^s + D\_max - 1$ ;
(22)      end
(23)       $B_i := \min(B_i^l, B_i^s)$ ;
(24)    end
(25)     $B_n := 0$ ;
(26)  end

```

Figure 7.11 Algorithm for computing the blocking factors.

According to the algorithm shown in Figure 7.11, the blocking factors of the tasks are computed as follows (note that one time unit is subtracted from each duration):

$$\begin{aligned}
 B_1^l &= 8 + 7 + 5 = 20 \\
 B_1^s &= 7 + 8 = 15 \quad \Rightarrow B_1 = 15
 \end{aligned}$$

$$\begin{aligned}
 B_2^l &= 7 + 5 = 12 \\
 B_2^s &= 7 + 6 + 3 = 16 \quad \Rightarrow B_2 = 12
 \end{aligned}$$

$$\begin{aligned}
B_3^l &= 5 \\
B_3^s &= 5 + 4 + 3 = 12 \quad ==> \quad B_3 = 5 \\
B_4^l &= B_4^s = 0 \quad ==> \quad B_4 = 0
\end{aligned}$$

To understand why the algorithm is pessimistic, note that B_2^l is computed by adding the duration of two critical sections both guarded by semaphore S_1 , which can never occur in practice.

7.6.4 IMPLEMENTATION CONSIDERATIONS

The implementation of the Priority Inheritance Protocol requires a slight modification of the kernel data structures associated with tasks and semaphores. First of all, each task must have a nominal priority and an active priority, which need to be stored in the Task Control Block (TCB). Moreover, in order to speed up the inheritance mechanism, it is convenient that each semaphore keeps track of the task holding the lock on it. This can be done by adding in the semaphore data structure a specific field, say *holder*, for storing the identifier of the holder. In this way, a task that is blocked on a semaphore can immediately identify the task that holds its lock for transmitting its priority. Similarly, transitive inheritance can be simplified if each task keeps track of the semaphore on which it is blocked. In this case, this information has to be stored in a field, say *lock*, of the Task Control Block. Assuming that the kernel data structures are extended as described above, the primitives *pi_wait* and *pi_signal* for realizing the Priority Inheritance Protocol can be defined as follows.

pi_wait(s)

- If semaphore s is free, it becomes locked and the name of the executing task is stored in the *holder* field of the semaphore data structure.
- If semaphore s is locked, the executing task is blocked on the s semaphore queue, the semaphore identifier is stored in the *lock* field of the TCB, and its priority is inherited by the task that holds s . If such a task is blocked on another semaphore, the transitivity rule is applied. Then, the ready task with the highest priority is assigned to the processor.

pi_signal(s)

- If the queue of semaphore s is empty (that is, no tasks are blocked on s), s is unlocked.

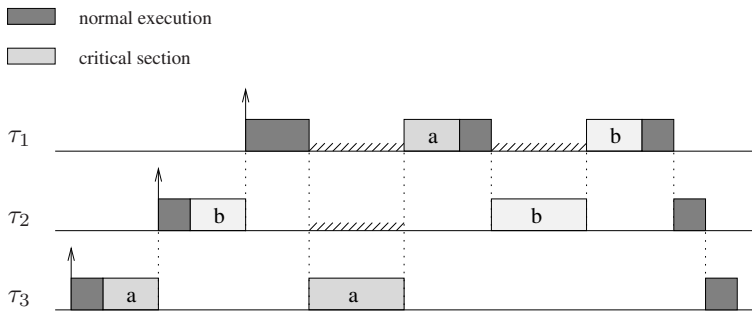


Figure 7.12 Example of chained blocking.

- If the queue of semaphore s is not empty, the highest-priority task in the queue is awakened, its identifier is stored in $s.holder$, the active priority of the executing task is updated and the ready task with the highest priority is assigned to the processor.

7.6.5 UNSOLVED PROBLEMS

Although the Priority Inheritance Protocol bounds the priority inversion phenomenon, the blocking duration for a task can still be substantial because a chain of blocking can be formed. Another problem is that the protocol does not prevent deadlocks.

CHAINED BLOCKING

Consider three tasks τ_1 , τ_2 and τ_3 with decreasing priorities that share two semaphores S_a and S_b . Suppose that τ_1 needs to sequentially access S_a and S_b , τ_2 accesses S_b , and τ_3 S_a . Also suppose that τ_3 locks S_a and it is preempted by τ_2 within its critical section. Similarly, τ_2 locks S_b and it is preempted by τ_1 within its critical section. The example is shown in Figure 7.12. In this situation, when attempting to use its resources, τ_1 is blocked for the duration of two critical sections, once to wait for τ_3 to release S_a and then to wait for τ_2 to release S_b . This is called a *chained blocking*. In the worst case, if τ_1 accesses n distinct semaphores that have been locked by n lower-priority tasks, τ_1 will be blocked for the duration of n critical sections.

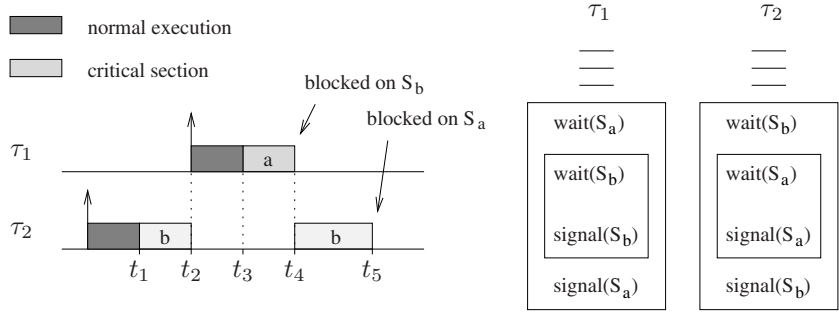


Figure 7.13 Example of deadlock.

DEADLOCK

Consider two tasks that use two semaphores in a nested fashion but in reverse order, as illustrated in Figure 7.13. Now suppose that, at time t_1 , τ_2 locks semaphore S_b and enters its critical section. At time t_2 , τ_1 preempts τ_2 before it can lock S_a . At time t_3 , τ_1 locks S_a , which is free, but then is blocked on S_b at time t_4 . At this time, τ_2 resumes and continues the execution at the priority of τ_1 . Priority inheritance does not prevent a deadlock, which occurs at time t_5 , when τ_2 attempts to lock S_a . Note, however, that the deadlock does not depend on the Priority Inheritance Protocol but is caused by an erroneous use of semaphores. In this case, the deadlock problem can be solved by imposing a total ordering on the semaphore accesses.

7.7 PRIORITY CEILING PROTOCOL

The Priority Ceiling Protocol (PCP) was introduced by Sha, Rajkumar, and Lehoczky [SRL90] to bound the priority inversion phenomenon and prevent the formation of deadlocks and chained blocking.

The basic idea of this method is to extend the Priority Inheritance Protocol with a rule for granting a lock request on a free semaphore. To avoid multiple blocking, this rule does not allow a task to enter a critical section if there are locked semaphores that could block it. This means that once a task enters its first critical section, it can never be blocked by lower-priority tasks until its completion.

In order to realize this idea, each semaphore is assigned a *priority ceiling* equal to the highest priority of the tasks that can lock it. Then, a task τ_i is allowed to enter a critical section only if its priority is higher than all priority ceilings of the semaphores currently locked by tasks other than τ_i .

7.7.1 PROTOCOL DEFINITION

The Priority Ceiling Protocol can be defined as follows:

- Each semaphore S_k is assigned a priority ceiling $C(S_k)$ equal to the highest priority of the tasks that can lock it. Note that $C(S_k)$ is a static value that can be computed off-line:

$$C(S_k) \stackrel{\text{def}}{=} \max_i \{P_i \mid S_k \in \sigma_i\}. \quad (7.13)$$

- Let τ_i be the task with the highest priority among all tasks ready to run; thus, τ_i is assigned the processor.
- Let S^* be the semaphore with the highest ceiling among all the semaphores currently locked by tasks other than τ_i and let $C(S^*)$ be its ceiling.
- To enter a critical section guarded by a semaphore S_k , τ_i must have a priority higher than $C(S^*)$. If $P_i \leq C(S^*)$, the lock on S_k is denied and τ_i is said to be blocked on semaphore S^* by the task that holds the lock on S^* .
- When a task τ_i is blocked on a semaphore, it transmits its priority to the task, say τ_j , that holds that semaphore. Hence, τ_j resumes and executes the rest of its critical section with the priority of τ_i . Task τ_j is said to *inherit* the priority of τ_i .
- In general, a task inherits the highest priority of the tasks blocked by it. That is, at every instant,

$$p_j(R_k) = \max\{P_j, \max_h \{P_h \mid \tau_h \text{ is blocked on } R_k\}\}. \quad (7.14)$$

- When τ_j exits a critical section, it unlocks the semaphore and the highest-priority task, if any, blocked on that semaphore is awakened. Moreover, the active priority of τ_j is updated as follows: if no other tasks are blocked by τ_j , p_j is set to the nominal priority P_j ; otherwise, it is set to the highest priority of the tasks blocked by τ_j , according to Equation (7.14).
- Priority inheritance is transitive; that is, if a task τ_3 blocks a task τ_2 , and τ_2 blocks a task τ_1 , then τ_3 inherits the priority of τ_1 via τ_2 .

EXAMPLE

In order to illustrate the Priority Ceiling Protocol, consider three tasks τ_1 , τ_2 , and τ_3 having decreasing priorities. The highest-priority task τ_1 sequentially accesses two critical sections guarded by semaphores S_A and S_B ; task τ_2 accesses only a critical section guarded by semaphore S_C ; whereas task τ_3 uses semaphore S_C and then makes a nested access to S_B . From tasks' resource requirements, all semaphores are assigned the following priority ceilings:

$$\begin{cases} C(S_A) = P_1 \\ C(S_B) = P_1 \\ C(S_C) = P_2. \end{cases}$$

Now suppose that events evolve as illustrated in Figure 7.14.

- At time t_0 , τ_3 is activated, and since it is the only task ready to run, it starts executing and later locks semaphore S_C .
- At time t_1 , τ_2 becomes ready and preempts τ_3 .
- At time t_2 , τ_2 attempts to lock S_C , but it is blocked by the protocol because P_2 is not greater than $C(S_C)$. Then, τ_3 inherits the priority of τ_2 and resumes its execution.
- At time t_3 , τ_3 successfully enters its nested critical section by locking S_B . Note that τ_3 is allowed to lock S_B because no semaphores are locked by other tasks.
- At time t_4 , while τ_3 is executing at a priority $p_3 = P_2$, τ_1 becomes ready and preempts τ_3 because $P_1 > p_3$.
- At time t_5 , τ_1 attempts to lock S_A , which is not locked by any task. However, τ_1 is blocked by the protocol because its priority is not higher than $C(S_B)$, which is the highest ceiling among all semaphores currently locked by the other tasks. Since S_B is locked by τ_3 , τ_3 inherits the priority of τ_1 and resumes its execution.
- At time t_6 , τ_3 exits its nested critical section, unlocks S_B , and, since τ_1 is awakened, τ_3 returns to priority $p_3 = P_2$. At this point, $P_1 > C(S_C)$; hence, τ_1 preempts τ_3 and executes until completion.
- At time t_7 , τ_1 is completed, and τ_3 resumes its execution at a priority $p_3 = P_2$.

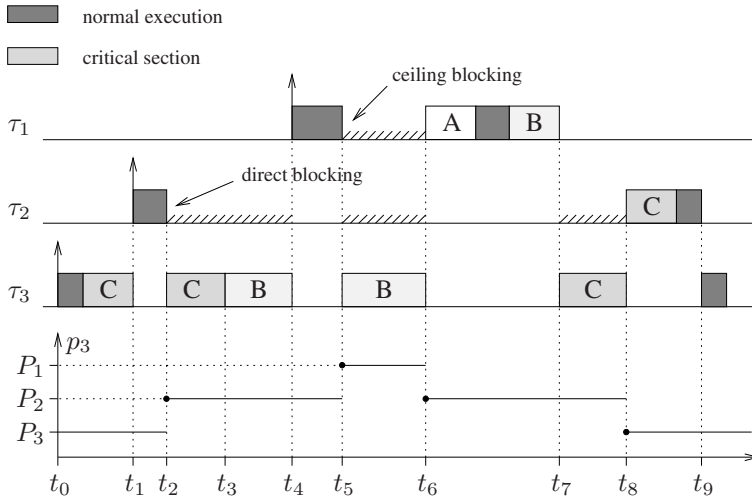


Figure 7.14 Example of Priority Ceiling Protocol.

- At time t_8 , τ_3 exits its outer critical section, unlocks S_C , and, since τ_2 is awakened, τ_3 returns to its nominal priority P_3 . At this point, τ_2 preempts τ_3 and executes until completion.
- At time t_9 , τ_2 is completed; thus, τ_3 resumes its execution.

Note that the Priority Ceiling Protocol introduces a third form of blocking, called *ceiling blocking*, in addition to direct blocking and push-through blocking caused by the Priority Inheritance Protocol. This is necessary for avoiding deadlock and chained blocking. In the previous example, a ceiling blocking is experienced by task τ_1 at time t_5 .

7.7.2 PROPERTIES OF THE PROTOCOL

The main properties of the Priority Ceiling Protocol are presented in this section. They are used to analyze the schedulability and compute the maximum blocking time of each task.

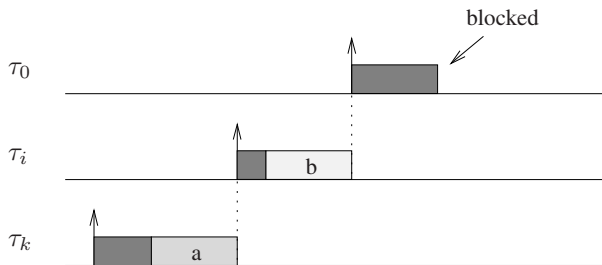


Figure 7.15 An absurd situation that cannot occur under the Priority Ceiling Protocol.

Lemma 7.6 *If a task τ_k is preempted within a critical section Z_a by a task τ_i that enters a critical section Z_b , then, under the Priority Ceiling Protocol, τ_k cannot inherit a priority higher than or equal to that of task τ_i until τ_i completes.*

Proof. If τ_k inherits a priority higher than or equal to that of task τ_i before τ_i completes, there must exist a task τ_0 blocked by τ_k , such that $P_0 \geq P_i$. This situation is shown in Figure 7.15. However, this leads to the contradiction that τ_0 cannot be blocked by τ_k . In fact, since τ_i enters its critical section, its priority must be higher than the maximum ceiling C^* of the semaphores currently locked by all lower-priority tasks. Hence, $P_0 \geq P_i > C^*$. But since $P_0 > C^*$, τ_0 cannot be blocked by τ_k , and the lemma follows. $\square$

Lemma 7.7 *The Priority Ceiling Protocol prevents transitive blocking.*

Proof. Suppose that a transitive block occurs; that is, there exist three tasks τ_1 , τ_2 , and τ_3 , with decreasing priorities, such that τ_3 blocks τ_2 and τ_2 blocks τ_1 . By the transitivity of the protocol, τ_3 will inherit the priority of τ_1 . However, this contradicts Lemma 7.6, which shows that τ_3 cannot inherit a priority higher than or equal to P_2 . Thus, the lemma follows. $\square$

Theorem 7.3 *The Priority Ceiling Protocol prevents deadlocks.*

Proof. Assuming that a task cannot deadlock by itself, a deadlock can only be formed by a cycle of tasks waiting for each other, as shown in Figure 7.16. In this situation,

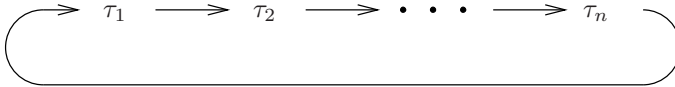


Figure 7.16 Deadlock among n tasks.

however, by the transitivity of the protocol, task τ_n would inherit the priority of τ_1 , which is assumed to be higher than P_n . This contradicts Lemma 7.6, and hence the theorem follows. $\square$

Theorem 7.4 (Sha, Rajkumar, Lehoczky) *Under the Priority Ceiling Protocol, a task τ_i can be blocked for at most the duration of one critical section.*

Proof. Suppose that τ_i is blocked by two lower-priority tasks τ_1 and τ_2 , where $P_2 < P_1 < P_i$. Let τ_2 enter its blocking critical section first, and let C_2^* be the highest-priority ceiling among all the semaphores locked by τ_2 . In this situation, if task τ_1 enters its critical section we must have that $P_1 > C_2^*$. Moreover, since we assumed that τ_i can be blocked by τ_2 , we must have that $P_i \leq C_2^*$. This means that $P_i \leq C_2^* < P_1$. This contradicts the assumption that $P_i > P_1$. Thus, the theorem follows. $\square$

7.7.3 BLOCKING TIME COMPUTATION

The evaluation of the maximum blocking time for each task can be computed based on the result of Theorem 7.4. According to this theorem, a task τ_i can be blocked for at most the duration of the longest critical section among those that can block τ_i . The set of critical sections that can block a task τ_i is identified by the following lemma.

Lemma 7.8 *Under the Priority Ceiling Protocol, a critical section $z_{j,k}$ (belonging to task τ_j and guarded by semaphore S_k) can block a task τ_i only if $P_j < P_i$ and $C(S_k) \geq P_i$.*

Proof. Clearly, if $P_j \geq P_i$, τ_i cannot preempt τ_j and hence cannot be blocked on $z_{j,k}$. Now assume $P_j < P_i$ and $C(S_k) < P_i$, and suppose that τ_i is blocked on $z_{j,k}$. We

	$S_a(P_1)$	$S_b(P_1)$	$S_c(P_2)$
τ_1	1	2	0
τ_2	0	9	3
τ_3	8	7	0
τ_4	6	5	4

Table 7.2 Three semaphores shared by four tasks.

show that this assumption leads to a contradiction. In fact, if τ_i is blocked by τ_j , its priority must be less than or equal to the maximum ceiling C^* among all semaphores locked by tasks other than τ_i . Thus, we have that $C(S_k) < P_i \leq C^*$. On the other hand, since C^* is the maximum ceiling among all semaphores currently locked by tasks other than τ_i , we have that $C^* \geq C(S_k)$, which leads to a contradiction and proves the lemma. $\square$

Using the result of Lemma 7.8, we can say that a task τ_i can only be blocked by critical sections belonging to lower priority tasks with a resource ceiling higher than or equal to P_i . That is,

$$\gamma_i = \{Z_{j,k} \mid (P_j < P_i) \text{ and } C(R_k) \geq P_i\}. \quad (7.15)$$

And since τ_i can be blocked at most once, the maximum blocking time τ_i can suffer is given by the duration of the longest critical section among those that can block τ_i . That is,

$$B_i = \max_{j,k} \{\delta_{j,k} - 1 \mid Z_{j,k} \in \gamma_i\}. \quad (7.16)$$

Consider the same example illustrated for the Priority Inheritance Protocol, reported in Table 7.2 for simplicity. According to Equation (7.15), we have:

$$\begin{aligned} \gamma_1 &= \{Z_{2b}, Z_{3a}, Z_{3b}, Z_{4a}, Z_{4b}\} \\ \gamma_2 &= \{Z_{3a}, Z_{3b}, Z_{4a}, Z_{4b}, Z_{4c}\} \\ \gamma_3 &= \{Z_{4a}, Z_{4b}, Z_{4c}\} \\ \gamma_4 &= \{\} \end{aligned}$$

Hence, based on Equation (7.16), tasks blocking factors are computed as follows:

$$\begin{cases} B_1 = \max(8, 7, 6, 5, 4) = 8 \\ B_2 = \max(7, 6, 5, 4, 3) = 7 \\ B_3 = \max(5, 4, 3) = 5 \\ B_4 = 0. \end{cases}$$

7.7.4 IMPLEMENTATION CONSIDERATIONS

The major implication of the Priority Ceiling Protocol in kernel data structures is that semaphores queues are no longer needed, since the tasks blocked by the protocol can be kept in the ready queue. In particular, whenever a task τ_i is blocked by the protocol on a semaphore S_k , the task τ_h that holds S_k inherits the priority of τ_i and it is assigned to the processor, whereas τ_i returns to the ready queue. As soon as τ_h unlocks S_k , p_h is updated and, if p_h becomes less than the priority of the first ready task, a context switch is performed.

To implement the Priority Ceiling Protocol, each semaphore S_k has to store the identifier of the task that holds the lock on S_k and the ceiling of S_k . Moreover, an additional field for storing the task active priority has to be reserved in the task control block. It is also convenient to have a field in the task control block for storing the identifier of the semaphore on which the task is blocked. Finally, the implementation of the protocol can be simplified if the system also maintains a list of currently locked semaphores, ordered by decreasing priority ceilings. This list is useful for computing the maximum priority ceiling that a task has to overcome to enter a critical section and for updating the active priority of tasks at the end of a critical section.

If the kernel data structures are extended as described above, the primitives *pc_wait* and *pc_signal* for realizing the Priority Ceiling Protocol can be defined as follows.

pc_wait(s)

- Find the semaphore S^* having the maximum ceiling C^* among all the semaphores currently locked by tasks other than the one in execution (τ_{exe}).
- If $p_{exe} \leq C^*$, transfer P_{exe} to the task that holds S^* , insert τ_{exe} in the ready queue, and execute the ready task (other than τ_{exe}) with the highest priority.
- If $p_{exe} > C^*$, or whenever s is unlocked, lock semaphore s , add s in the list of currently locked semaphores and store τ_{exe} in $s.holder$.

pc_signal(s)

- Extract s from the list of currently locked semaphores.
- If no other tasks are blocked by τ_{exe} , set $p_{exe} = P_{exe}$, else set p_{exe} to the highest priority of the tasks blocked by τ_{exe} .

- Let p^* be the highest priority among the ready tasks. If $p_{exe} < p^*$, insert τ_{exe} in the ready queue and execute the ready task (other than τ_{exe}) with the highest priority.

7.8 STACK RESOURCE POLICY

The Stack Resource Policy (SRP) is a technique proposed by Baker [Bak91] for accessing shared resources. It extends the Priority Ceiling Protocol (PCP) in three essential points:

1. It allows the use of multi-unit resources.
2. It supports dynamic priority scheduling.
3. It allows the sharing of runtime stack-based resources.

From a scheduling point of view, the essential difference between the PCP and the SRP is on the time at which a task is blocked. Whereas under the PCP a task is blocked at the time it makes its first resource request, under the SRP a task is blocked at the time it attempts to preempt. This early blocking slightly reduces concurrency but saves unnecessary context switches, simplifies the implementation of the protocol, and allows the sharing of runtime stack resources.

7.8.1 DEFINITIONS

Before presenting the formal description of the SRP we introduce the following definitions.

PRIORITY

Each task τ_i is assigned a priority p_i that indicates the importance (that is, the urgency) of τ_i with respect to the other tasks in the system. Priorities can be assigned to tasks either statically or dynamically. At any time t , $p_a > p_b$ means that the execution of τ_a is more important than that of τ_b ; hence, τ_b can be delayed in favor of τ_a . For example, priorities can be assigned to tasks based on Rate Monotonic (RM) or Earliest Deadline First (EDF).

PREEMPTION LEVEL

Besides a priority p_i , a task τ_i is also characterized by a *preemption level* π_i . The preemption level is a static parameter, assigned to a task at its creation time and associated with all instances of that task. The essential property of preemption levels is that a task τ_a can preempt another task τ_b only if $\pi_a > \pi_b$. This is also true for priorities. Hence, the reason for distinguishing preemption levels from priorities is that preemption levels are fixed values that can be used to predict potential blocking also in the presence of dynamic priority schemes. The general definition of preemption level used to prove all properties of the SRP requires that

if τ_a arrives after τ_b and τ_a has higher priority than τ_b , then τ_a must have a higher preemption level than τ_b .

Under EDF scheduling, the previous condition is satisfied if preemption levels are ordered inversely with respect to the order of relative deadlines; that is,

$$\pi_i > \pi_j \iff D_i < D_j.$$

To better illustrate the difference between priorities and preemption levels, consider the example shown in Figure 7.17. Here we have two tasks τ_1 and τ_2 , with relative deadlines $D_1 = 10$ and $D_2 = 5$, respectively. Being $D_2 < D_1$, we define $\pi_1 = 1$ and $\pi_2 = 2$. Since $\pi_1 < \pi_2$, τ_1 can never preempt τ_2 ; however, τ_1 may have a priority higher than that of τ_2 . In fact, under EDF, the priority of a task is dynamically assigned based on its absolute deadline. For example, in the case illustrated in Figure 7.17a, the absolute deadlines are such that $d_2 < d_1$; hence, τ_2 will have higher priority than τ_1 . On the other hand, as shown in Figure 7.17b, if τ_2 arrives a time $r_1 + 6$, absolute deadlines are such that $d_2 > d_1$; hence, τ_1 will have higher priority than τ_2 . Note that although τ_1 has a priority higher than τ_2 , τ_2 cannot be preempted. This happens because, when $d_1 < d_2$ and $D_1 > D_2$, τ_1 always starts before τ_2 ; thus, it does not need to preempt τ_2 .

In the following, it is assumed that tasks are ordered by decreasing preemption levels, so that $i < j \iff \pi_i > \pi_j$. This also means that $D_1 < D_2 < \dots < D_n$.

RESOURCE UNITS

Each resource R_k is allowed to have N_k units that can be concurrently accessed by multiple tasks. This notion generalizes the classical single-unit resource, which can be accessed by one task at the time under mutual exclusion. A resource with N_k units can be concurrently accessed by N_k different tasks, if each task requires one unit.

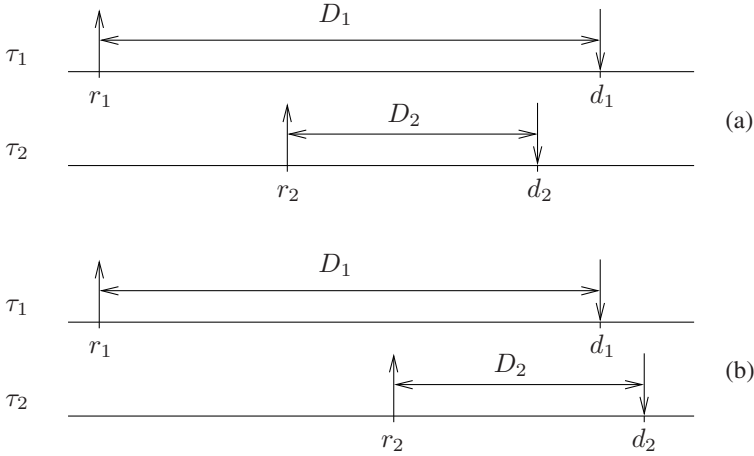


Figure 7.17 Although $\pi_2 > \pi_1$, under EDF p_2 can be higher than p_1 (a) or lower than p_1 (b).

In general, n_k denotes the number of currently available units for R_k , meaning that $N_k - n_k$ units are locked. If $n_k = 0$, a task requiring 3 units of R_k is blocked until n_k becomes greater than or equal to 3.

RESOURCE REQUIREMENTS

When entering a critical section $z_{i,k}$ guarded by a multi-unit semaphore S_k , a task τ_i must specify the number of units it needs by calling a *wait*(S_k, r), where r is the number of requested units. When exiting the critical section, τ_i must call a *signal*(S_k), which releases all the r units.

The maximum number of units that can be simultaneously requested by τ_i to R_k is denoted by $\mu_i(R_k)$, which can be derived off-line by analyzing the task code. It is clear that if τ_i requires more units than are available, that is, if $\mu_i(R_k) > n_k$, then τ_i blocks until n_k becomes greater than or equal to $\mu_i(R_k)$.

RESOURCE CEILING

Each resource R_k (or semaphore S_k) is assigned a *ceiling* $C_{Rk}(n_k)$ equal to the highest preemption level of the tasks that could be blocked on R_k if issuing their maximum request. Hence, the ceiling of a resource is a dynamic value, which is a function of the

	D_i	π_i	$\mu_i(R_1)$	$\mu_i(R_2)$	$\mu_i(R_3)$
τ_1	5	3	1	0	1
τ_2	10	2	2	1	3
τ_3	20	1	3	1	1

Table 7.3 Task parameters and resource requirements.

	$C_R(3)$	$C_R(2)$	$C_R(1)$	$C_R(0)$
R_1	0	1	2	3
R_2	-	-	0	2
R_3	0	2	2	3

Table 7.4 Resource ceilings as a function of the number of available units. Dashes identify impossible cases.

units of R_k that are currently available. That is,

$$C_{R_k}(n_k) = \max\{\pi_i \mid \mu_i(R_k) > n_k\}.$$

If all units of R_k are available, that is, if $n_k = N_k$, then $C_{R_k}(N_k) = 0$.

To better clarify this concept, consider the following example, where three tasks (τ_1 , τ_2 , τ_3) share three resources (R_1 , R_2 , R_3), consisting of three, one, and three units, respectively. All tasks parameters – relative deadlines, preemption levels, and resource requirements – are shown in Table 7.3.

Based on these requirements, the current ceilings of the resources as a function of the number n_k of available units are reported in Table 7.4 (dashes identify impossible cases).

Let us compute, for example, the ceiling of resource R_1 when only two units (out of three) are available. From Table 7.3, we see that the only task that could be blocked in this condition is τ_3 because it requires three units of R_1 ; hence, $C_{R_1}(2) = \pi_3 = 1$. If only one unit of R_1 is available, the tasks that could be blocked are τ_2 and τ_3 ; hence, $C_{R_1}(1) = \max(\pi_2, \pi_3) = 2$. Finally, if none of the units of R_1 is available, all three tasks could be blocked on R_1 ; hence, $C_{R_1}(0) = \max(\pi_1, \pi_2, \pi_3) = 3$.

SYSTEM CEILING

The resource access protocol adopted in the SRP also requires a *system ceiling*, Π_s , defined as the maximum of the current ceilings of all the resources; that is,

$$\Pi_s = \max_k \{C_{R_k}\}.$$

Notice that Π_s is a dynamic parameter that can change every time a resource is accessed or released by a task.

7.8.2 PROTOCOL DEFINITION

The key idea of the SRP is that when a task needs a resource that is not available, it is blocked at the time it attempts to preempt rather than later. Moreover, to prevent multiple priority inversions, a task is not allowed to start until the resources currently available are sufficient to meet the maximum requirement of every task that could preempt it. Using the definitions introduced in the previous paragraph, this is achieved by the following preemption test:

SRP Preemption Test: A task is not permitted to preempt until its priority is the highest among those of all the tasks ready to run, and its preemption level is higher than the system ceiling.

If the ready queue is ordered by decreasing priorities, the preemption test can be simply performed by comparing the preemption level $\pi(\tau)$ of the ready task with the highest priority (the one at the head of the queue) with the system ceiling. If $\pi(\tau) > \Pi_s$, task τ is executed, otherwise it is kept in the ready queue until Π_s becomes less than $\pi(\tau)$. The condition $\pi(\tau) > \Pi_s$ has to be tested every time Π_s may decrease; that is, every time a resource is released.

OBSERVATIONS

The implications that the use of the SRP has on tasks' execution can be better understood through the following observations:

- Passing the preemption test for task τ ensures that the resources that are currently available are sufficient to satisfy the maximum requirement of task τ and the maximum requirement of every task that could preempt τ . This means that once τ starts executing, it will never be blocked for resource contention.

- Although the preemption test for a task τ is performed before τ starts executing, resources are not allocated at this time but only when requested.
- A task can be blocked by the preemption test even though it does not require any resource. This is needed to avoid unbounded priority inversion.
- Blocking at preemption time, rather than at access time, introduces more pessimism and could create unnecessary blocking (as under HLP), but it decreases the number of context switches, reduces the run-time overhead, and simplifies the implementation of the protocol.
- The preemption test has the effect of imposing priority inheritance; that is, an executing task that holds a resource modifies the system ceiling and resists preemption as though it inherits the priority of any tasks that might need that resource. Note that this effect is accomplished without modifying the priority of the task.

EXAMPLE

In order to illustrate how the SRP works, consider the task set already described in Table 7.3. The structure of the tasks is shown in Figure 7.18, where $wait(R_k, r)$ denotes the request of r units of resource R_k , and $signal(R_k)$ denotes their release. The current ceilings of the resources have already been shown in Table 7.4, and a possible EDF schedule for this task set is depicted in Figure 7.19. In this figure, the fourth timeline reports the variation of the system ceiling, whereas the numbers along the schedule denote resource indexes.

At time t_0 , τ_3 starts executing and the system ceiling is zero because all resources are completely available. When τ_3 enters its first critical section, it takes the only unit of R_2 ; thus, the system ceiling is set to the highest preemption level among the tasks that could be blocked on R_2 (see Table 7.4); that is, $\Pi_s = \pi_2 = 2$. As a consequence, τ_2 is blocked by the preemption test and τ_3 continues to execute. Note that when τ_3 enters its nested critical section (taking all units of R_1), the system ceiling is raised to $\Pi_s = \pi_1 = 3$. This causes τ_1 to be blocked by the preemption test.

As τ_3 releases R_1 (at time t_2), the system ceiling becomes $\Pi_s = 2$; thus, τ_1 preempts τ_3 and starts executing. Note that once τ_1 is started, it is never blocked during its execution, because the condition $\pi_1 > \Pi_s$ guarantees that all the resources needed by τ_1 are available. As τ_1 terminates, τ_3 resumes the execution and releases resource R_2 . As R_2 is released, the system ceiling returns to zero and τ_2 can preempt τ_3 . Again, once τ_2 is started, all the resources it needs are available; thus, τ_2 is never blocked.

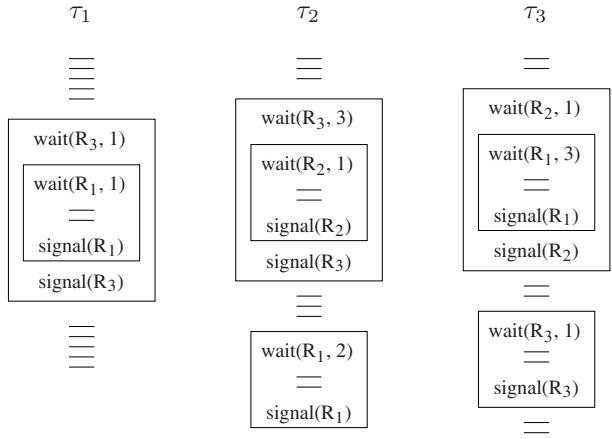


Figure 7.18 Structure of the tasks in the SRP example.

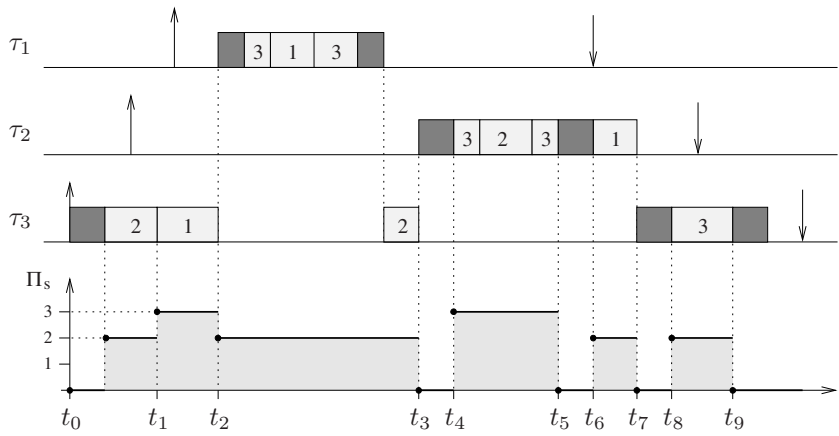


Figure 7.19 Example of a schedule under EDF and SRP. Numbers on tasks' execution denote the resource indexes.

7.8.3 PROPERTIES OF THE PROTOCOL

The main properties of the Stack Resource Policy are presented in this section. They will be used to analyze the schedulability and compute the maximum blocking time of each task.

Lemma 7.9 *If the preemption level of a task τ is greater than the current ceiling of a resource R , then there are sufficient units of R available to*

1. *meet the maximum requirement of τ and*
2. *meet the maximum requirement of every task that can preempt τ .*

Proof. Assume $\pi(\tau) > C_R$, but suppose that the maximum request of τ for R cannot be satisfied. Then, by definition of current ceiling of a resource, we have $C_R \geq \pi(\tau)$, which is a contradiction.

Assume $\pi(\tau) > C_R$, but suppose that there exists a task τ_H that can preempt τ such that the maximum request of τ_H for R cannot be satisfied. Since τ_H can preempt τ , it must be $\pi(\tau_H) > \pi(\tau)$. Moreover, since the maximum request of τ_H for R cannot be satisfied, by definition of current ceiling of a resource, we have $C_R \geq \pi(\tau_H) > \pi(\tau)$, which contradicts the assumption. $\square$

Theorem 7.5 (Baker) *If no task τ is permitted to start until $\pi(\tau) > \Pi_s$, then no task can be blocked after it starts.*

Proof. Let N be the number of tasks that can preempt a task τ and assume that no task is permitted to start until its preemption level is greater than Π_s . The thesis will be proved by induction on N .

If $N = 0$, there are no tasks that can preempt τ . If τ is started when $\pi(\tau) > \Pi_s$, Lemma 7.9 guarantees that all the resources required by τ are available when τ preempts; hence, τ will execute to completion without blocking.

If $N > 0$, suppose that τ is preempted by τ_H . If τ_H is started when $\pi(\tau_H) > \Pi_s$, Lemma 7.9 guarantees that all the resources required by τ_H are available when τ_H preempts. Since any task that preempts τ_H also preempts τ , the induction hypothesis guarantees that τ_H executes to completion without blocking, as will any task that preempts τ_H , transitively. When all the tasks that preempted τ complete, τ can resume its

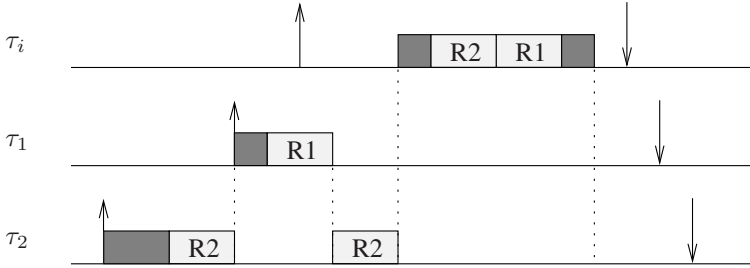


Figure 7.20 An absurd situation that cannot occur under SRP.

execution without blocking, since the higher-priority tasks released all resources and when τ started the resources available were sufficient to meet the maximum request of τ . $\square$

Theorem 7.6 (Baker) *Under the Stack Resource Policy, a task τ_i can be blocked for at most the duration of one critical section.*

Proof. Suppose that τ_i is blocked for the duration of two critical sections shared with two lower-priority tasks, τ_1 and τ_2 . Without loss of generality, assume $\pi_2 < \pi_1 < \pi_i$. This can happen only if τ_1 and τ_2 hold two different resources (such as R_1 and R_2) and τ_2 is preempted by τ_1 inside its critical section. This situation is depicted in Figure 7.20. This immediately yields to a contradiction. In fact, since τ_1 is not blocked by the preemption test, we have $\pi_1 > \Pi_s$. On the other hand, since τ_i is blocked, we have $\pi_i \leq \Pi_s$. Hence, we obtain that $\pi_i < \pi_1$, which contradicts the assumption. $\square$

Theorem 7.7 (Baker) *The Stack Resource Policy prevents deadlocks.*

Proof. By Theorem 7.5, a task cannot be blocked after it starts. Since a task cannot be blocked while holding a resource, there can be no deadlock. $\square$

7.8.4 BLOCKING TIME COMPUTATION

The maximum blocking time that a task can experience with the SRP is the same as the one that can be experienced with the Priority Ceiling Protocol. Theorem 7.6, in fact, guarantees that under the SRP a task τ_i can be blocked for at most the duration of one critical section among those that can block τ_i . Lemma 7.8, proved for the PCP, can be easily extended to the SRP; thus a critical section $Z_{j,k}$ belonging to task τ_j and guarded by semaphore S_k can block a task τ_i only if $\pi_j < \pi_i$ and $\max(C_{S_k}) \geq \pi_i$. Note that under the SRP, the ceiling of a semaphore is a dynamic variable, so we have to consider its maximum value; that is, the one corresponding to a number of units currently available equal to zero.

Hence, a task τ_i can only be blocked by critical sections belonging to tasks with preemption level lower than π_i and guarded by semaphores with maximum ceiling higher than or equal to π_i . That is,

$$\gamma_i = \{Z_{j,k} \mid (\pi_j < \pi_i) \text{ and } C_{S_k}(0) \geq \pi_i\}. \quad (7.17)$$

And since τ_i can be blocked at most once, the maximum blocking time τ_i can suffer is given by the duration of the longest critical section among those that can block τ_i . That is,

$$B_i = \max_{j,k} \{\delta_{j,k} - 1 \mid Z_{j,k} \in \gamma_i\}. \quad (7.18)$$

7.8.5 SHARING RUNTIME STACK

Another interesting implication deriving from the use of the SRP is that it supports stack sharing among tasks. This is particularly convenient for those applications consisting of a large number of tasks, dedicated to acquisition, monitoring, and control activities. In conventional operating systems, each process must have a private stack space, sufficient to store its context (that is, the content of the CPU registers) and its local variables. A problem with these systems is that, if the number of tasks is large, a great amount of memory may be required for the stacks of all the tasks.

For example, consider four tasks τ_1 , τ_2 , τ_3 , and τ_4 , with preemption levels 1, 2, 2, and 3, respectively (3 being the highest preemption level). Figure 7.21 illustrates a possible evolution of the stacks, assuming that each task is allocated its own stack space, equal to its maximum requirement. At time t_1 , τ_1 starts executing; τ_2 preempts at time t_2 and completes at time t_3 , allowing τ_1 to resume. At time t_4 , τ_1 is preempted by τ_3 , which in turn is preempted by τ_4 at time t_5 . At time t_6 , τ_4 completes and τ_3 resumes. At time t_7 , τ_3 completes and τ_1 resumes.

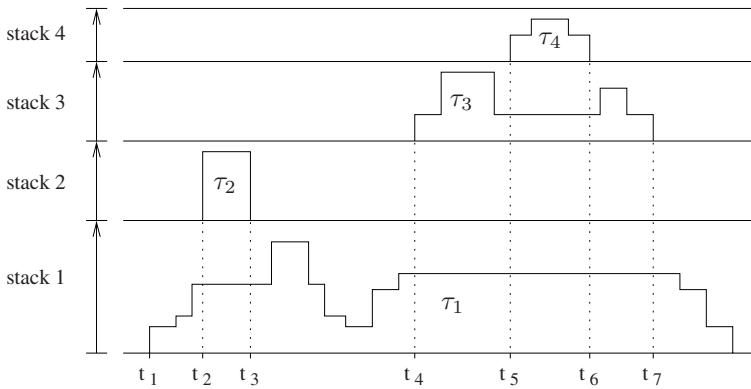


Figure 7.21 Possible evolution with one stack per task.

Note that the top of each process stack varies during the process execution, while the storage region reserved for each stack remains constant and corresponds to the distance between two horizontal lines. In this case, the total storage area that must be reserved for the application is equal to the sum of the stack regions dedicated to each process.

However, if all tasks are independent or use the SRP to access shared resources, then they can share a single stack space. In this case, when a task τ is preempted by a task τ' , τ maintains its stack and the stack of τ' is allocated immediately above that of τ . Figure 7.22 shows a possible evolution of the previous task set when a single stack is allocated to all tasks.

Under the SRP, stack overlapping without interpenetration is a direct consequence of Theorem 7.5. In fact, since a task τ can never be blocked once started, its stack can never be penetrated by the ones belonging to tasks with lower preemption levels, which can resume only after τ is completed.

Note that the stack space between the two upper horizontal lines (which is equivalent to the minimum stack between τ_2 and τ_3) is no longer needed, since τ_2 and τ_3 have the same preemption level, so they can never occupy stack space at the same time. In general, the higher the number of tasks with the same preemption level, the larger stack saving.

For example, consider an application consisting of 100 tasks distributed on 10 preemption levels, with 10 tasks for each level, and suppose that each task needs up to 10 Kbytes of stack space. Using a stack per task, 1000 Kbytes would be required. On

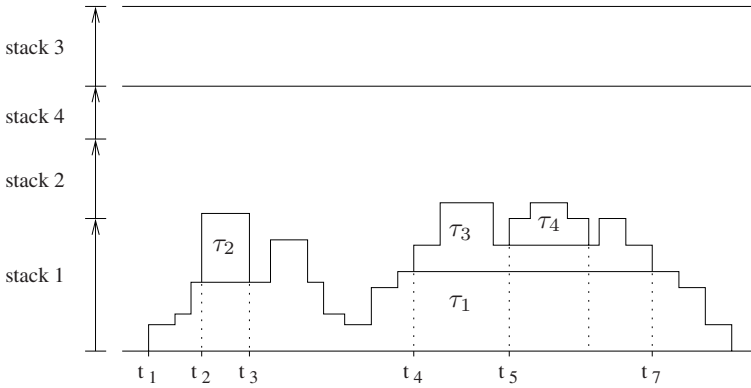


Figure 7.22 Possible evolution with a single stack for all tasks.

the contrary, using a single stack, only 100 Kbytes would be sufficient, since no more than one task per preemption level could be active at one time. Hence, in this example we would save 900 Kbytes; that is, 90%. In general, when tasks are distributed on k preemption levels, the space required for a single stack is equal to the sum of the largest request on each level.

7.8.6 IMPLEMENTATION CONSIDERATIONS

The implementation of the SRP is similar to that of the PCP, but the locking operations (*srp_wait* and *srp_signal*) are simpler, since a task can never be blocked when attempting to lock a semaphore. When there are no sufficient resources available to satisfy the maximum requirement of a task, the task is not permitted to preempt and is kept in the ready queue.

To simplify the preemption test, all the ceilings of the resources (for any number of available units) can be precomputed and stored in a table. Moreover, a stack can be used to keep track of the system ceiling. When a resource R is allocated, its current state, n_R , is updated and if $C_R(n_R) > \Pi_s$, then Π_s is set to $C_R(n_R)$. The old values of n_R and Π_s are pushed onto the stack. When resource R is released, the values of Π_s and n_R are restored from the stack. If the restored system ceiling is lower than the previous value, the preemption test is executed on the ready task with the highest priority to check whether it can preempt. If the preemption test is passed, a context switch is performed; otherwise, the current task continues its execution.

7.9 SCHEDULABILITY ANALYSIS

This section explains how to verify the feasibility of a periodic task set in the presence of shared resources. All schedulability tests presented in Chapter 4 for independent tasks can be extended to include blocking terms, whose values depend on the specific concurrency control protocol adopted in the schedule.

In general, all the extended tests guarantee one task τ_i at the time, by inflating its computation time C_i by the blocking factor B_i . In addition, all the guarantee tests that were necessary and sufficient under preemptive scheduling become only sufficient in the presence of blocking factors, since blocking conditions are derived in worst-case scenarios that differ for each task and could never occur simultaneously.

Liu and Layland test for Rate Monotonic. A set of periodic tasks with blocking factors and relative deadlines equal to periods is schedulable by RM if

$$\forall i = 1, \dots, n \quad \sum_{h: P_h > P_i} \frac{C_h}{T_h} + \frac{C_i + B_i}{T_i} \leq i(2^{1/i} - 1). \quad (7.19)$$

Liu and Layland test for EDF. A set of periodic tasks with blocking factors and relative deadlines equal to periods is schedulable by EDF if

$$\forall i = 1, \dots, n \quad \sum_{h: P_h > P_i} \frac{C_h}{T_h} + \frac{C_i + B_i}{T_i} \leq 1. \quad (7.20)$$

Hyperbolic Test. Using the Hyperbolic Bound, a task set with blocking factors and relative deadlines equal to periods is schedulable by RM if

$$\forall i = 1, \dots, n \quad \prod_{h: P_h > P_i} \left(\frac{C_h}{T_h} + 1 \right) \left(\frac{C_i + B_i}{T_i} + 1 \right) \leq 2. \quad (7.21)$$

Response Time Analysis. Under blocking conditions, the response time of a generic task τ_i with a fixed priority can be computed by the following recurrent relation:

$$\begin{cases} R_i^{(0)} = C_i + B_i \\ R_i^{(s)} = C_i + B_i + \sum_{h: P_h > P_i} \left\lceil \frac{R_i^{(s-1)}}{T_h} \right\rceil C_h. \end{cases} \quad (7.22)$$

Workload Analysis. Similarly, using the workload analysis, a task set with blocking factors is schedulable by a fixed priority assignment if

$$\forall i = 1, \dots, n \quad \exists t \in \mathcal{TS}_i : B_i + W_i(t) \leq t. \quad (7.23)$$

where set $\mathcal{TS}_i$ has been defined by equation (4.21).

Processor Demand Criterion. The Processor Demand Criterion in the presence of blocking terms has been extended by Baruah [Bar06], using the concept of *Blocking Function* $B(L)$, defined as the largest amount of time for which a task with relative deadline $\leq L$ may be blocked by a task with relative deadline $> L$.

If δ_{jh} denotes the maximum length of time for which τ_j holds a resource that is also needed by τ_h , the blocking function can be computed as follows:

$$B(L) = \max \{ \delta_{j,h} \mid (D_j > L) \text{ and } (D_h \leq L) \}. \quad (7.24)$$

Then, a task set can be scheduled by EDF if $U < 1$ and

$$\forall L \in \mathcal{D} \quad B(L) + g(0, L) \leq L. \quad (7.25)$$

where $\mathcal{D}$ is the set of all absolute deadlines no greater than a certain point in time, given by the minimum between the hyperperiod H and the following expression:

$$\max \left(D_{max}, \frac{\sum_{i=1}^n (T_i - D_i) U_i}{1 - U} \right).$$

where $D_{max} = \max\{D_1, \dots, D_n\}$.

7.10 SUMMARY

The concurrency control protocols presented in this chapter can be compared with respect to several characteristics. Table 7.5 provides a qualitative evaluation of the algorithms in terms of priority assignment, number of blocking, level of pessimism, instant of blocking, programming transparency, deadlock prevention, and implementation complexity. Here, transparency refers to the impact of the protocol on the programming interface. A transparent protocol (like NPP and PIP) can be implemented without modifying the task code, that is, exploiting the same primitives used by classical semaphores. This means that legacy applications developed using classical semaphores (prone to priority inversion) can also be executed under a transparent protocol. This feature makes such protocols attractive for commercial operating systems (like VxWorks), where predictability can be improved without introducing new kernel primitives. On the contrary, protocols that use resource ceilings (as HLP, PCP and SRP) need an additional system call to specify such values in the application.

	priority	Num. of blocking	pessimism	blocking instant	transparency	deadlock prevention	implementation
NPP	any	1	high	on arrival	YES	YES	easy
HLP	fixed	1	medium	on arrival	NO	YES	easy
PIP	fixed	α_i	low	on access	YES	NO	hard
PCP	fixed	1	medium	on access	NO	YES	medium
SRP	any	1	medium	on arrival	NO	YES	easy

Table 7.5 Evaluation summary of resource access protocols.

Exercises

7.1 Verify whether the following task set is schedulable by the Rate-Monotonic algorithm. Apply the processor utilization approach first, and then the Response Time Analysis:

	C_i	T_i	B_i
τ_1	4	10	5
τ_2	3	15	3
τ_3	4	20	0

7.2 Consider three periodic tasks τ_1 , τ_2 , and τ_3 (having decreasing priority) that share three resources, A , B , and C , accessed using the Priority Inheritance Protocol. Compute the maximum blocking time B_i for each task, knowing that the longest duration $\delta_{i,R}$ for a task τ_i on resource R is given in the following table (there are no nested critical sections):

	A	B	C
τ_1	3	0	3
τ_2	3	4	0
τ_3	4	3	6

- 7.3 Solve the same problem described in Exercise 7.2 when the resources are accessed by the Priority Ceiling Protocol.
- 7.4 For the task set described in Exercise 7.2, illustrate the situation produced by RM + PIP in which task τ_2 experiences its maximum blocking time.
- 7.5 Consider four periodic tasks τ_1 , τ_2 , τ_3 , and τ_4 (having decreasing priority) that share five resources, A , B , C , D , and E , accessed using the Priority Inheritance Protocol. Compute the maximum blocking time B_i for each task, knowing that the longest duration $\delta_{i,R}$ for a task τ_i on resource R is given in the following table (there are no nested critical sections):

	A	B	C	D	E
τ_1	3	6	10	0	7
τ_2	0	0	8	0	0
τ_3	0	4	0	8	14
τ_4	7	0	9	0	11

- 7.6 Solve the same problem described in Exercise 7.5 when the resources are accessed by the Priority Ceiling Protocol.
- 7.7 For the task set described in Exercise 7.5, illustrate the situation produced by RM + PIP in which task τ_2 experiences its maximum blocking time.
- 7.8 Consider three tasks τ_1 , τ_2 , and τ_3 that share three multi-unit resources, A , B , and C , accessed using the Stack Resource Policy. Resources A and B have three units, whereas C has two units. Compute the ceiling table for all the resources based on the following task characteristics:

	D_i	μ_A	μ_B	μ_C
τ_1	5	1	0	1
τ_2	10	2	1	2
τ_3	20	3	1	1

